debug

```
#include <bits/stdc++.h>

using namespace std;
using ll = long long;
#define sz(a) (int)(a).size()
#define all(a) (a).begin(), (a).end()

template<typename A>
string to_string(A v) {
    string s = "{";
    for (auto x : v) {
        if (sz(s) > 1) {
            s += ", ";
        }
        s += to_string(x);
    }
    return s += "}";
}

void debug_out() {
    cerr << "\n";
}

template<typename T, typename... U>
void debug_out(const T& x, const U&... args) {
    cerr << " " << to_string(x);
    debug_out(args...);
}

#define debug(...) cerr << "[" << #__VA_ARGS__ << "]:", debug_out(__VA_ARGS__)

namespace IO {
    // TODO: fwrite
    const int N = 1 << 20;

    struct fistream {
        char buf[N], *s, *t;

        fistream() : s(buf), t(buf) {}

        char getchar() {
            if (s == t) {
                s = buf;
                t = buf + fread(buf, 1, N, stdin);
            }
            return s == t ? EOF : *s++;
```

```
        }
    } in;

    template<typename U>
    fistream& operator>>(fistream& st, U& x) {
        char c = st.getchar();
        int t = 1;
        x = 0;
        while (!isdigit(c)) {
            if (c == '-') {
                t *= -1;
            }
            c = st.getchar();
        }
        while (isdigit(c)) {
            x = x * 10 + (c ^ 48);
            c = st.getchar();
        }
        x *= t;
        return st;
    }
}

namespace Mod {
    // constexpr int mod = 998244353;
    // constexpr int mod = 1000000007;
    constexpr int mod = -1;

    void reduce(int& x) {
        (x >= mod) ? x -= mod : 0;
    }

    int exp(int a, int x = mod - 2) {
        int p = 1;
        for (; x; x >>= 1) {
            if (x & 1) {
                p = p * (ll)a % mod;
            }
            a = a * (ll)a % mod;
        }
        return p;
    }
}

namespace Comb {
    using namespace Mod;

    vector<int> inv, fac, ifac;

    // n: max value involved
    void comb_init(int n) {
        inv = fac = ifac = vector(n + 1, 0);
        inv[1] = 1;
```

```
        for (int i = 2; i <= n; ++i) {
            inv[i] = (mod - mod / i) * (ll)inv[mod % i] % mod;
        }
        fac[0] = ifac[0] = 1;
        for (int i = 1; i <= n; ++i) {
            fac[i] = fac[i - 1] * (ll)i % mod;
            ifac[i] = ifac[i - 1] * (ll)inv[i] % mod;
        }
    }

    int C(int n, int m) {
        return 0 <= m and m <= n ? fac[n] * (ll)ifac[m] % mod * ifac[n - m] %
mod : 0;
    }
}

namespace DS {
    template<typename T>
    struct table {
        vector<vector<T>> f;
        function<T(T, T)> C;

        table(const vector<T>& a, function<T(T, T)> C): C(C) {
            f = vector(32 - __builtin_clz(sz(a)), vector<T>());
            f[0] = a;
            for (int k = 1; k < sz(f); ++k) {
                f[k] = vector<T>(sz(a) - (1 << k) + 1);
                for (int i = 0; i < sz(f[k]); ++i) {
                    f[k][i] = C(f[k - 1][i], f[k - 1][i + (1 << (k - 1))]);
                }
            }
        }

        T F(int l, int r) {
            int k = 32 - __builtin_clz(r - l) - 1;
            return C(f[k][l], f[k][r - (1 << k)]);
        }
    };

    template<typename T>
    struct fenwick {
        vector<T> f;
        function<void(T&, T)> E;

        fenwick(int n, function<void(T&, T)> E): f(n), E(E) {}

        fenwick(const vector<T>& a, function<void(T&, T)> E): f(a), E(E) {
            for (int i = 0; i < sz(f); ++i) {
                E(f[i | (i + 1)], f[i]);
            }
        }

        void add(int i, T x) {
```

```
        for (; i < sz(f); i |= i + 1) {
            E(f[i], x);
        }
    }

    T sum(int i) {
        T x{};
        for (; i; i &= i - 1) {
            E(x, f[i - 1]);
        }
        return x;
    }
};

template<typename T, typename U>
struct sumt {
    int cnt, n, rt;
    vector<int> ls, rs;
    vector<T> f;
    vector<U> g;

    void pushup(int u) {
        f[u] = f[ls[u]] + f[rs[u]]; // Override!
    }

    void pushtag(int u, const U& v) {
        f[u] += v; // Override!
        g[u] += v; // Override!
    }

    void pushdown(int u) {
        // code: operator bool() const {}
        if (g[u]) {
            pushtag(ls[u], g[u]);
            pushtag(rs[u], g[u]);
            g[u] = U{};
        }
    }

    int build(int l, int r, const vector<T>& a) {
        int u = cnt++;
        if (r - l == 1) {
            f[u] = a[l];
            return u;
        }
        int mid = (l + r) / 2;
        ls[u] = build(l, mid, a);
        rs[u] = build(mid, r, a);
        pushup(u);
        return u;
    }

    sumt(const vector<T>& a): cnt(0), n(sz(a)), ls(n * 2), rs(n * 2), f(n *
```

```
2), g(n * 2) {
            rt = build(0, n, a);
        }

        void add(int x, int y, const U& v, int u, int l, int r) {
            if (x >= r or y <= l) {
                return;
            }
            if (x <= l and r <= y) {
                pushtag(u, v);
                return;
            }
            int mid = (l + r) / 2;
            pushdown(u);
            add(x, y, v, ls[u], l, mid);
            add(x, y, v, rs[u], mid, r);
            pushup(u);
        }

        void add(int x, int y, const U& v) {
            add(x, y, v, rt, 0, n);
        }

        T sum(int x, int y, int u, int l, int r) {
            if (x >= r or y <= l) {
                return T{};
            }
            if (x <= l and r <= y) {
                return f[u];
            }
            int mid = (l + r) / 2;
            pushdown(u);
            return sum(x, y, ls[u], l, mid) + sum(x, y, rs[u], mid, r);
        }

        T sum(int x, int y) {
            return sum(x, y, rt, 0, n);
        }
    };

    struct dsu {
        vector<int> f;

        dsu(int n): f(n) {
            iota(all(f), 0);
        }

        int F(int u) {
            return u == f[u] ? u : f[u] = F(f[u]);
        }

        int operator()(int u) {
            return F(u);
```

```
        }
    };
}

namespace NT {
    vector<int> prime, lp;

    void sieve(int n) {
        lp = vector(n + 1, -1);
        for (int i = 2; i <= n; ++i) {
            if (!~lp[i]) {
                lp[i] = sz(prime);
                prime.push_back(i);
            }
            for (int j = 0; j <= lp[i] and i * prime[j] <= n; ++j) {
                lp[i * prime[j]] = j;
            }
        }
    }
}

// namespace Grp {}
// namespace Mat {}
// namespace Rng {}
// namespace Str {}
// namespace Geo {}
// namespace PN {}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

}
```

checker linux

```
    system("g++ A.cpp -o A");
    int tt;
    cin >> tt;
    for (int i = 1; i <= tt; ++i) {
        string s = to_string(i);
        string command_1 = "./A < " + s + ".in > " + s + ".out";
        string command_2 = "diff " + s + ".out " + s + ".ans";
        system(command_1.c_str());
        if (system(command_2.c_str())) cerr << "Wrong answer on testcase " << i
<< '\n';
        else cerr << "Accepted on testcase " << i << '\n';
        cerr << "################################\n";
    }
```

gcc

```
#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")
#pragma GCC
target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,avx2,tune=native")
```

sam

```
struct SuffixM {
    int p;
    vector<int> fa, len;
    vector<array<int, 2>> ch;

    SuffixM(int n) : p(1), fa(n, -1), len(n, 0), ch(n, {-1, -1}) {}

    int Split(int u, int c) {
        int v = ch[u][c];
        if (len[v] == len[u] + 1) return v;
        int w = p++;
        len[w] = len[u] + 1;
        fa[w] = fa[v];
        fa[v] = w;
        ch[w] = ch[v];
        for (; ~u && ch[u][c] == v; u = fa[u]) ch[u][c] = w;
        return w;
    }

    int Extend(int u, int c) {
        if (~ch[u][c]) return Split(u, c);
        int v = p++;
        len[v] = len[u] + 1;
        for (; ~u && !~ch[u][c]; u = fa[u]) ch[u][c] = v;
        fa[v] = ~u ? Split(u, c) : 0;
        return v;
    }
};
```

poly

```
template<typename T = int>
int power(int a, T x = mod - 2) {
    int p = 1;
    while (x) {
        if (x & 1) p = (long long) p * a % mod;
        x >>= 1;
        a = (long long) a * a % mod;
    }
    return p;
}

vector<int> rev;
vector<int> w = {0, 1};

void dft(vector<int> &a) {
    int n = a.size();
    if (n != (int) rev.size()) {
        rev.resize(n);
        for (int i = 0; i < n; ++i) rev[i] = (rev[i / 2] / 2) | (i & 1) * (n /
2);
    }
    int nw = w.size();
    if (nw < n) {
        w.resize(n);
        for (; nw < n; nw *= 2) {
            const int wn = power(G, (mod - 1) / (nw * 2));
            for (int i = nw / 2; i < nw; ++i) w[i * 2 + 1] = (long long) (w[i *
2] = w[i]) * wn % mod;
        }
    }
    for (int i = 0; i < n; ++i) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int m = 1; m < n; m *= 2) for (int i = 0; i < n; i += m * 2) for (int
j = 0; j < m; ++j) {
        const int u = a[i | j];
        const int v = (long long) w[m | j] * a[i | m | j] % mod;
        int x = u + v;
        if (x >= mod) x -= mod;
        a[i | j] = x;
        x = u - v;
        if (x < 0) x += mod;
        a[i | m | j] = x;
    }
}

void idft(vector<int> &a) {
    int n = a.size(), inv = n == 1 ? 1 : mod - mod / n;
    reverse(a.begin() + 1, a.end());
    dft(a);
    for (int i = 0; i < n; ++i) a[i] = (long long) a[i] * inv % mod;
```

```
}

vector<int> operator*(vector<int> a, vector<int> b) {
    int mx = a.size() + b.size() - 1;
    int n = 1 << (32 - __builtin_clz(mx - 1));
    a.resize(n, 0);
    dft(a);
    b.resize(n, 0);
    dft(b);
    vector<int> c(n);
    for (int i = 0; i < n; ++i) c[i] = (long long) a[i] * b[i] % mod;
    idft(c);
    c.resize(mx);
    return c;
}
```

mt19937 rng rand

```
mt19937 rgen(__builtin_ia32_rdtsc());
uniform_int_distribution<u64> distr(1, -1);
auto rng = bind(distr, rgen);
```

xorshift

```
unsigned long long xorshift(unsigned long long x) {
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    return x;
}
```

fread

```
constexpr int buf_size = 1 << 20;

struct fastream {
    char buf[buf_size], *s, *t;

    fastream() : s(buf), t(buf) {}

    char getchar() {
        if (s == t) t = (s = buf) + fread(buf, 1, buf_size, stdin);
        return s == t ? EOF : *s++;
    }
} fin;

template<typename U>
fastream& operator>>(fastream& fst, U& x) {
    char c = fst.getchar(), t = 1;
    x = 0;
    for (; !isdigit(c); c = fst.getchar()) if (c == '-') t *= -1;
    for (; isdigit(c); c = fst.getchar()) x = x * 10 + (c ^ 48);
    x *= t;
    return fst;
}
```

rmqtable

```
struct rmqtable {
    static constexpr int L = 20;
    int N;
    vector<vector<pair<int, int>>> f;

    rmqtable(const vector<int>& a) : N(a.size()), f(L, vector<pair<int, int>>
(N)) {
        for (int i = 0; i < N; ++i) f[0][i] = {a[i], i};
        for (int l = 0; l + 1 < L; ++l) for (int i = 0; i + (2 << l) <= N; ++i)
f[l + 1][i] = max(f[l][i], f[l][i + (1 << l)]);
    }

    pair<int, int> rmq(int l, int r) {
        int k = 32 - __builtin_clz(r - l) - 1;
        return max(f[k][l], f[k][r - (1 << k)]);
    }
};
```

fhqtreap

```
mt19937 rng(__builtin_ia32_rdtsc());

struct fhqtreap {
    int p;
    vector<int> ls, rs, X, L, R, sz;

    fhqtreap(int N) : p(1), ls(N), rs(N), X(N), L(N), R(N), sz(N, 0) {}

    int node(int x, int l, int r) {
        int u = p++;
        X[u] = x;
        L[u] = l;
        R[u] = r;
        sz[u] = r - l;
        return u;
    }

    void pushup(int u) {
        sz[u] = R[u] - L[u] + sz[ls[u]] + sz[rs[u]];
    }

    int leftmost(int u) {
        while (ls[u]) u = ls[u];
        return u;
    }

    int rightmost(int u) {
        while (rs[u]) u = rs[u];
        return u;
    }

    int merge(int u, int v) {
        if (!u or !v) return u ^ v;
        int r;
        if (int(rng() % (sz[u] + sz[v])) < sz[u]) {
            rs[u] = merge(rs[u], v);
            pushup(u);
            r = u;
        }
        else {
            ls[v] = merge(u, ls[v]);
            pushup(v);
            r = v;
        }
        return r;
    }

    void split_s(int r, int k, int& u, int& v) {
        if (!r) {
```

```
            u = v = 0;
            return;
        }
        if (k >= sz[ls[r]] + (R[r] - L[r])) {
            u = r;
            split_s(rs[r], k - sz[ls[r]] - (R[r] - L[r]), rs[u], v);
            pushup(u);
        }
        else {
            v = r;
            split_s(ls[r], k, u, ls[v]);
            pushup(v);
        }
    }

    void split_x(int r, int x, int& u, int& v) {
        if (!r) {
            u = v = 0;
            return;
        }
        if (x < X[r]) {
            v = r;
            split_x(ls[r], x, u, ls[v]);
            pushup(v);
        }
        else {
            u = r;
            split_x(rs[r], x, rs[u], v);
            pushup(u);
        }
    }
};
```

max-flow

```
long long flow = 0;
{
    int m = e.size();
    vector<int> head(n, -1), to(m * 2), nxt(m * 2);
    vector<long long> cap(m * 2);
    int p = 0;
    auto adde = [&](int u, int v, int w) {
        to[p] = v;
        cap[p] = w;
        nxt[p] = head[u];
        head[u] = p++;
    };
    for (const auto& [u, v, w] : e) {
        adde(u, v, w);
        adde(v, u, 0);
    }
    assert(p == m * 2);
    while (true) {
        vector<int> dis(n, -1);
        queue<int> q;
        dis[s] = 0;
        q.push(s);
        while (q.size()) {
            int u = q.front();
            q.pop();
            for (int i = head[u]; ~i; i = nxt[i]) if (cap[i] and !~dis[to[i]])
{
                dis[to[i]] = dis[u] + 1;
                q.push(to[i]);
            }
        }
        if (!~dis[t]) break;
        vector<int> cur = head;
        auto dfs = [&](auto dfs, int u, long long c)->long long {
            if (u == t) return c;
            long long r = c, f;
            for (int &i = cur[u]; ~i; i = nxt[i]) if (cap[i] and dis[to[i]] ==
dis[u] + 1 and (f = dfs(dfs, to[i], min(cap[i], r)))) {
                cap[i] -= f;
                cap[i ^ 1] += f;
                r -= f;
                if (!r) break;
            }
            return c - r;
        };
        flow += dfs(dfs, s, (long long) 1e18);
```

```
        }
    }
```

min-cost max-flow

```
class Net {
private:
    struct Edge {
        int to, nxt, cap, w;
    };

    int n;
    vector<Edge> e;
    vector<int> g;

public:
    int dist, flow, cost;

    Net(int n): n(n), g(n, -1), dist(0), flow(0), cost(0) {}

    void Add(int u, int v, int c, int w) {
        e.push_back({v, g[u], c, w});
        g[u] = e.size() - 1;
        e.push_back({u, g[v], 0, -w});
        g[v] = e.size() - 1;
    }

    void Solve(int s, int t) {
        while (true) {
            vector<int> dep(n, (int)1e9);
            priority_queue<pair<int, int>> que;
            dep[t] = 0;
            que.emplace(0, t);
            while (que.size()) {
                auto [d, u] = que.top();
                que.pop();
                if (-d != dep[u]) continue;
                for (int i = g[u]; ~i; i = e[i].nxt) if (e[i ^ 1].cap &&
dep[e[i].to] > dep[u] - e[i].w) {
                    dep[e[i].to] = dep[u] - e[i].w;
                    que.emplace(-dep[e[i].to], e[i].to);
                }
            }
            if (dep[s] == (int)1e9) break;
            for (int u = 0; u < n; ++u) for (int i = g[u]; ~i; i = e[i].nxt)
e[i].w += dep[e[i].to] - dep[u];
            dist += dep[s];
            vector<int> vis(n, false);
            function<int(int, int)> Dfs = [&](int u, int c) {
                vis[u] = true;
                if (u == t) return c;
                int r = c, f;
                for (int i = g[u]; ~i; i = e[i].nxt) if (e[i].cap &&
!vis[e[i].to] && !e[i].w && (f = Dfs(e[i].to, min(c, e[i].cap)))) {
```

```
                    e[i].cap -= f;
                    e[i ^ 1].cap += f;
                    r -= f;
                    if (!r) break;
                }
                return c - r;
            };
            int sum = 0;
            while (true) {
                sum += Dfs(s, (int)1e9);
                if (!vis[t]) break;
                vector<int>(n, false).swap(vis);
            }
            flow += sum;
            cost += sum * dist;
        }
    }
};
```

exgcd

```
template<typename T>
void exgcd(T a, T b, T& x, T& y, T& g) {
    if (a == 0) {
        x = 0;
        y = 1;
        g = b;
        return;
    }
    //              (b % a) * y + (a * x) = g
    //      <=>     a * (x - (b / a) * y) + b * y = g
    exgcd(b % a, a, y, x, g);
    x -= (b / a) * y;
}
// assert(a * x + b * y == g);
// assert(-b / g < x and x < b / g and -a / g < y and y < a / g);
```

geometry

```
struct P {
    ld x, y;

    friend P operator+(P a, P b) { return P{a.x + b.x, a.y + b.y}; }
    friend P operator-(P a, P b) { return P{a.x - b.x, a.y - b.y}; }
    friend P operator*(P a, ld k) { return P{a.x * k, a.y * k}; }
    friend ld Cross(P a, P b) { return a.x * b.y - a.y * b.x; }
};
```

inter half-planes

```
const int kL = 1000;
vector<P> c{P{-kL, -kL}, P{kL, -kL}, P{kL, kL}, P{-kL, kL}};
auto Inter = [&](P a, P b, P u, P v) {
    P lhs = b - a, rhs = v - u, d = u - a;
    return a + lhs * (Cross(rhs, d) / Cross(rhs, lhs));
};
auto Cut = [&](P a, P b) {
    int n = c.size();
    vector<int> inside(n);
    for (int i = 0; i < n; ++i) inside[i] = (Cross(a - c[i], b - c[i]) >= 0);
    vector<P> new_c;
    for (int i = 0; i < n; ++i) {
        if (inside[i]) new_c.push_back(c[i]);
        if (inside[i] != inside[i + 1 == n ? 0 : i + 1])
new_c.push_back(Inter(c[i], c[i + 1 == n ? 0 : i + 1], a, b));
    }
    swap(c, new_c);
};
int tt;
cin >> tt;
while (tt--) {
    int n;
    cin >> n;
    vector<P> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i].x >> a[i].y;
    for (int i = 0; i < n; ++i) Cut(a[i], a[i + 1 == n ? 0 : i + 1]);
}
ld s = 0;
for (int i = 1; i + 1 < (int)c.size(); ++i) s += Cross(c[i] - c[0], c[i + 1] -
c[0]);
s /= 2;
cout << fixed << setprecision(3) << s << '\n';
```

km

```
class KM {  // See if variables could be int
public:
    int n;
    vector<int> mat;
    ll sum;
    vector<ll> lv, rv;
    vector<vector<ll>> e;

    KM(int n): n(n), mat(n + 1, -1), sum(0), lv(n), rv(n + 1, 0), e(n,
vector<ll>(n, -(ll)1e18)) {}

    void Add(int u, int v, ll w) { e[u][v] = w; }

    void Solve() {
        for (int u = 0; u < n; ++u) lv[u] = *max_element(e[u].begin(),
e[u].end());
        for (int s = 0; s < n; ++s) {
            vector<int> vis(n + 1, false);
            vector<int> pre(n + 1, -1);
            vector<ll> rmn(n + 1, (ll)1e18);
            mat[n] = s;
            int w = n;
            for (int u = -1; ~(u = mat[w]); ) {
                vis[w] = true;
                pair<ll, int> mn((ll)1e18, -1);
                ll min_gap, gap;
                for (int v = 0; v <= n; ++v) if (!vis[v]) {
                    if ((gap = lv[u] + rv[v] - e[u][v]) < rmn[v]) {
                        rmn[v] = gap;
                        pre[v] = w;
                    }
                    mn = min(mn, make_pair(rmn[v], v));
                }
                tie(min_gap, w) = mn;
                for (int v = 0; v <= n; ++v) {
                    if (vis[v]) {
                        rv[v] += min_gap;
                        lv[mat[v]] -= min_gap;
                    }
                    else rmn[v] -= min_gap;
                }
            }
            for (; w != n; w = pre[w]) mat[w] = mat[pre[w]];
        }
        rv[n] = 0;
        sum = accumulate(lv.begin(), lv.end(), 0ll) + accumulate(rv.begin(),
rv.end(), 0ll);
```

```
    }
};
```

stoer wagner

```
int mn = (int)1e9;
vector<int> exist(n, true);
for (int i = 0; i < n - 1; ++i) {
    vector<int> w(n, 0);
    vector<int> o;
    vector<int> vis = exist;
    for (int j = 0; j < n - i; ++j) {
        int x = -1;
        for (int u = 0; u < n; ++u) if (vis[u] && (!~x || w[u] > w[x])) x = u;
        vis[x] = false;
        o.push_back(x);
        for (int u = 0; u < n; ++u) if (vis[u]) w[u] += e[x][u];
    }
    int s = *(o.end() - 2), t = o.back();
    mn = min(mn, w[t]);
    exist[t] = false;
    for (int u = 0; u < n; ++u) {
        e[s][u] += e[t][u];
        e[u][s] += e[u][t];
    }
}
```